**LOW-LEVEL DESIGN DOCUMENT**

RAG-Based Customer Support Assistant with HITL

| Property | Value |
|---|---|
| Project Name | RAG-Based Customer Support Assistant |
| Document Type | Low-Level Design (LLD) |
| Version | 1.0 |
| Date | April 2026 |
| Author | Abhishek Sambhaji Bhosale |
| Intern ID | IN226053602 |

## 1. Module Architecture

### 1.1 Module Dependency Overview

The application is organized into seven specialized modules. Below is the dependency hierarchy showing how modules interact:

```
main.py (Entry Point)
  ├── document_processor.py  ← PyPDFLoader, RecursiveCharacterTextSplitter
  ├── embedding_manager.py   ← OpenAIEmbeddings, ChromaDB
  └── graph_builder.py       ← Compiles LangGraph StateGraph
      ├── graph_state.py     ← QueryState TypedDict definition
      ├── graph_nodes.py     ← retrieve_node, generate_node, escalate_node
      |   └── hitl_manager.py ← HITLManager (ticket CRUD)
      └── routing_logic.py   ← should_escalate(), detect_intent()


app.py (Streamlit UI)
  └── Imports CustomerSupportBot from main.py
```

### 1.2 Module Descriptions

| Module | File | Responsibility | Key Classes/Functions |
|---|---|---|---|
| Document Processor | src/document_processor.py | PDF loading and text chunking | DocumentProcessor |
| Embedding Manager | src/embedding_manager.py | Vector store creation and retrieval | EmbeddingManager |
| Graph State | src/graph_state.py | State schema definition | QueryState (TypedDict) |
| Graph Nodes | src/graph_nodes.py | LangGraph node implementations | GraphNodes |

| Module | File | Responsibility | Key Classes/Functions |
|---|---|---|---|
| Routing Logic | src/routing_logic.py | Escalation and intent decision logic | should_escalate(), detect_intent() |
| Graph Builder | src/graph_builder.py | LangGraph workflow compilation | build_graph() |
| HITL Manager | src/hitl_manager.py | Escalation queue management | HITLManager |

## 2. Class-Level Design

### 2.1 DocumentProcessor Class

**Constructor Parameters**

| Parameter | Type | Default | Description |
|---|---|---|---|
| pdf_path | str | Required | Absolute or relative path to the input PDF file |
| chunk_size | int | 500 | Maximum character count per chunk |
| chunk_overlap | int | 50 | Character overlap between consecutive chunks |

**Method Specifications**

| Method | Input | Output | Exception Handling |
|---|---|---|---|
| load_documents() | None | List[Document] | Raises FileNotFoundError if PDF path invalid |
| chunk_documents(docs) | List[Document] | List[Document] | Returns empty list [] if input is empty |

```
class DocumentProcessor:
    def __init__(self, pdf_path: str, chunk_size: int = 500, chunk_overlap: int = 50):
        self.pdf_path = pdf_path
        self.text_splitter = RecursiveCharacterTextSplitter(
            chunk_size=chunk_size,
            chunk_overlap=chunk_overlap
        )

    def load_documents(self) -> List[Document]:
        loader = PyPDFLoader(self.pdf_path)
        return loader.load()
```

| Method | Input | Output | Exception Handling |
|---|---|---|---|

```
def chunk_documents(self, documents: List[Document]) -> List[Document]:
    if not documents:
        return []
    return self.text_splitter.split_documents(documents)
```

**2.2 EmbeddingManager Class**

```
class EmbeddingManager:
    def __init__(self, persist_directory: str = './chroma_db'):
        self.embeddings = OpenAIEmbeddings(model='text-embedding-3-small')
        self.persist_directory = persist_directory

    def create_vector_store(self, chunks: List[Document]) -> Chroma:
        # Creates and persists ChromaDB from chunks
        return Chroma.from_documents(chunks, self.embeddings,
                                     persist_directory=self.persist_directory)

    def load_vector_store(self) -> Chroma:
        # Loads existing ChromaDB; raises ValueError if not found
        if not os.path.exists(self.persist_directory):
            raise ValueError('Vector store not found. Run ingestion first.')
        return Chroma(persist_directory=self.persist_directory,
                      embedding_function=self.embeddings)
```

### 2.3 QueryState (TypedDict)

The QueryState TypedDict is the central data structure that flows between all LangGraph nodes. Every node reads from and writes to this shared state object.

| Field | Type | Populated By | Description |
|---|---|---|---|
| query | str | User input | Original user question, unchanged throughout workflow |
| retrieved_docs | List[str] | retrieve_node | List of raw page_content strings from ChromaDB |
| context | str | retrieve_node | Concatenated string of all retrieved chunks |
| response | str | generate_node / escalate_node | Final response to show the user |
| confidence_score | float | generate_node | Float 0.0–1.0; drives escalation routing |
| needs_escalation | bool | escalate_node | True if routed to human agent |
| intent | Literal[...] | Initial processing | One of: general, product, return, technical, unknown |
| human_feedback | Optional[str] | HITL dashboard | Agent response if escalation resolved |

### 2.4 GraphNodes Class

| Node Method | Node Name in Graph | Input Fields Read | Output Fields Written |
|---|---|---|---|
| retrieve_node(state) | retrieve | query | retrieved_docs, context |
| generate_node(state) | generate | context, query | response, confidence_score |

| Node Method | Node Name in Graph | Input Fields Read | Output Fields Written |
|---|---|---|---|
| escalate_node(state) | escalate | query, response, confidence_score | needs_escalation=True, response (updated) |

```
def _calculate_confidence(self, response: str, retrieved_docs: List) -> float:
    score = 0.5   # baseline
    if retrieved_docs:                    score += 0.3
    if len(response) > 50:                 score += 0.1
    hedge_words = ['maybe', 'perhaps', 'i think', 'not sure']
    if not any(w in response.lower() for w in hedge_words): score += 0.1
    return min(score, 1.0)
```

**2.5 Routing Logic Module**

```
ESCALATION_KEYWORDS = ['complaint', 'damaged', 'broken', 'unacceptable', 'legal']


def should_escalate(state: QueryState) -> Literal['escalate', 'respond']:
    if not state['retrieved_docs']:          return 'escalate'  # No context
    if state['confidence_score'] < 0.6:      return 'escalate'  # Low confidence
    if state['intent'] == 'unknown':         return 'escalate'  # Unknown intent
    if any(kw in state['query'].lower()       # Complex keyword
           for kw in ESCALATION_KEYWORDS):    return 'escalate'
    return 'respond'


INTENT_KEYWORDS = {
    'product':   ['specifications', 'features', 'compatible', 'dimensions'],
    'return':    ['return', 'refund', 'exchange', 'money back'],
    'technical': ['error', 'not working', 'issue', 'broken', 'bug'],
```

```
}
```

### 2.6 HITLManager Class

```
class HITLManager:
    def create_escalation_ticket(self, state: QueryState) -> dict:
        ticket = {
            'ticket_id': f"ESC_{datetime.now().strftime('%Y%m%d_%H%M%S')}",
            'query': state['query'],
            'llm_response': state['response'],
            'confidence_score': state['confidence_score'],
            'context_used': state['context'][:500],  # truncated
            'timestamp': datetime.now().isoformat(),
            'status': 'pending',
            'human_response': None
        }
        self._save_ticket(ticket)
        return ticket

    def resolve_escalation(self, ticket_id: str, human_response: str) -> None:
        tickets = self._load_tickets()
        for t in tickets:
            if t['ticket_id'] == ticket_id:
                t['status'] = 'resolved'
                t['human_response'] = human_response
        self._save_tickets(tickets)
```

## 3. Data Structures

### 3.1 Escalation Ticket Schema (JSON)

```
{
  "ticket_id": "ESC_20260422_143022",
  "query": "I want to file a complaint about a damaged product",
  "llm_response": "I apologize for the inconvenience. I'm escalating...",
  "confidence_score": 0.45,
  "context_used": "Return policy states: 30 days from purchase...",
  "timestamp": "2026-04-22T14:30:22",
  "status": "pending",
  "human_response": null
}
```

### 3.2 ChromaDB Internal Embedding Format

```
{
  "id": "uuid-f34a9e...",
  "embedding": [0.123, -0.456, 0.789, ...],  // 1536 float dimensions
  "document": "Chunked text content (500 chars max)",
  "metadata": {
    "source": "customer_support.pdf",
    "page": 3,
    "chunk_index": 12,
    "total_chunks": 87
  }
}
```

### 3.3 API Response Schema

```
# Returned by CustomerSupportBot.process_query()
```

```
{
  "query": "What is your return policy?",
  "response": "You can return products within 30 days of purchase...",
  "needs_escalation": False,
  "confidence": 0.90,
  "ticket_id": None   # Populated only when needs_escalation=True
}
```

## 4. LangGraph Workflow Design

### 4.1 Graph Definition Code

```
from langgraph.graph import StateGraph, END


def build_graph(nodes: GraphNodes) -> CompiledGraph:
    workflow = StateGraph(QueryState)


    # Register nodes
    workflow.add_node('retrieve', nodes.retrieve_node)
    workflow.add_node('generate', nodes.generate_node)
    workflow.add_node('escalate', nodes.escalate_node)


    # Define edges
    workflow.set_entry_point('retrieve')
    workflow.add_edge('retrieve', 'generate')
    workflow.add_conditional_edges(
        'generate',
        should_escalate,
        { 'respond': END, 'escalate': 'escalate' }
    )
    workflow.add_edge('escalate', END)


    return workflow.compile(checkpointer=MemorySaver())
```

### 4.2 Node and Edge Summary

| Node | Entry Condition | Key Operations | Exit Condition |
|---|---|---|---|
| retrieve | Always (entry point) | similarity_search(query, k=4) | Always → generate |

| Node | Entry Condition | Key Operations | Exit Condition |
|---|---|---|---|
| generate | After retrieve | LLM invoke, confidence calc | conditional: respond or escalate |
| escalate | confidence < 0.6 or trigger | create_ticket, format message | Always → END |

## 5. Error Handling Strategy

| Error Scenario | Detection Method | Recovery Action | User Impact |
|---|---|---|---|
| PDF file not found | FileNotFoundError on load | Raise exception with clear message | Ingestion fails; admin notified |
| Empty document chunks | len(chunks) == 0 check | Log warning; abort ingestion | Knowledge base empty |
| Embedding API timeout | requests.Timeout exception | Retry 3x with exponential backoff | Delayed ingestion |
| No relevant chunks retrieved | len(retrieved_docs) == 0 | Trigger escalation path | Query escalated to human |
| LLM generation timeout | asyncio.TimeoutError (10s) | Retry once; then escalate | Response delayed or escalated |
| JSON file corruption | json.JSONDecodeError | Backup file; reinitialize queue | Pending tickets may be lost |

## 6. Testing Strategy

### 6.1 Unit Test Cases

| Test ID | Test Case | Input Query | Expected Output |
|---|---|---|---|
| UT-01 | Standard return query | What is your return policy? | needs_escalation=False, confidence > 0.6 |
| UT-02 | Product spec query | Tell me about product dimensions | needs_escalation=False, intent=product |
| UT-03 | Technical issue | My device won't turn on | needs_escalation=False, intent=technical |
| UT-04 | Out-of-scope query | What is the meaning of life? | needs_escalation=True, intent=unknown |
| UT-05 | Complex keyword trigger | I want to file a complaint | needs_escalation=True |
| UT-06 | Empty retrieval | CEO email address | needs_escalation=True, retrieved_docs=[] |

### 6.2 Sample Integration Test

```
# tests/test_bot.py
@pytest.mark.asyncio
async def test_normal_query_does_not_escalate():
    bot = CustomerSupportBot()
    bot.initialize()
    result = await bot.process_query('What is your return policy?')
    assert result['needs_escalation'] is False
    assert 'return' in result['response'].lower()
    assert result['confidence'] > 0.6


@pytest.mark.asyncio
async def test_complaint_query_escalates():
    bot = CustomerSupportBot()
    bot.initialize()
    result = await bot.process_query('I want to file a complaint')
    assert result['needs_escalation'] is True
    assert result.get('ticket_id') is not None
```

## 7. Deployment Configuration

### 7.1 Environment Variables

| Variable | Required | Default | Description |
|---|---|---|---|
| OPENAI_API_KEY | Yes | None | OpenAI API key for embeddings and LLM calls |
| CHROMA_PERSIST_DIR | No | ./chroma_db | Directory path for ChromaDB persistence |
| PDF_PATH | No | ./data/customer_support.pdf | Path to knowledge base PDF |
| ESCALATION_FILE | No | ./escalations.json | Path for HITL ticket queue file |

### 7.2 Setup and Run Commands

```
# 1. Clone repository
git clone https://github.com/yourusername/rag-customer-support.git
cd rag-customer-support

# 2. Create virtual environment
python -m venv venv
source venv/bin/activate      # Windows: venv\Scripts\activate

# 3. Install dependencies
pip install -r requirements.txt

# 4. Configure environment
cp .env.example .env
# Edit .env and add your OPENAI_API_KEY
```

```
# 5. Run document ingestion (one-time setup)
python main.py --ingest

# 6. Launch Streamlit UI
streamlit run app.py
```

## 8. Project File Structure

```
rag-customer-support/
 ├── src/
 |   ├── __init__.py
 |   ├── document_processor.py   # PDF loading + text chunking
 |   ├── embedding_manager.py    # ChromaDB + OpenAI embeddings
 |   ├── graph_state.py          # QueryState TypedDict schema
 |   ├── graph_nodes.py          # retrieve, generate, escalate nodes
 |   ├── routing_logic.py        # should_escalate(), detect_intent()
 |   ├── graph_builder.py        # LangGraph StateGraph compilation
 |   └── hitl_manager.py         # JSON ticket queue operations
 ├── data/
 |   └── customer_support.pdf    # Knowledge base document
 ├── tests/
 |   └── test_bot.py             # pytest unit + integration tests
 ├── docs/
 |   ├── HLD.docx
 |   ├── LLD.docx
 |   └── Technical_Documentation.docx
 ├── main.py                     # CustomerSupportBot core class
 ├── app.py                      # Streamlit web interface
 ├── requirements.txt            # Python dependencies
 ├── .env.example                # Environment variable template
 └── README.md                   # Setup and usage guide
```